

C# 프로그래밍

동명대학교 게임학부 · 2026년 1학기

Lecture 5

게임으로 배우는 클래스 활용

숫자 맞추기에서 행맨까지

강영민

동명대학교 게임학부

Contents

1 강의 개요	2
1.1 학습 목표	2
1.2 강의 구성	2
2 A부: 숫자 맞추기 게임	3
2.1 Step A-1: MyNumber 클래스 만들기	3
2.2 Step A-2: Checker 클래스 — 클래스 간 협력	4
2.3 Step A-3: 상속과 오버라이딩으로 게임 완성	5
3 B부: 숫자 야구 게임	10
3.1 Step B-1: SecretNum 클래스	10
3.2 Step B-2: Checker 클래스 — 스트라이크/볼 판정	11
3.3 Step B-3: GameManager로 게임 완성	11
4 C부: 행맨 (Hangman) 게임	14
4.1 Step C-1: WordClass — 단어 저장과 알파벳 검사	14
4.2 Step C-2: HangmanDrawing — 그림 그리기	15
4.3 Step C-3: CheckWord — 빈칸 관리와 글자 채우기	15
4.4 Step C-4: GameManager로 행맨 완성	16
5 상속과 다형성 — 세 게임을 관통하는 핵심 원리	19
5.1 상속이 사용된 곳: Step A-3의 GameNumber	19
5.2 클래스 간 협력이 사용된 곳: 모든 게임	19
5.3 다형성의 기초: “같은 이름, 다른 동작”	20
6 핵심 개념 정리	22
6.1 이번 강의에서 다룬 OOP 개념	22
6.2 세 게임의 공통 구조	22

1. 강의 개요

1.1. 학습 목표

이번 강의에서는 Lecture 4에서 학습한 클래스를 활용하여 세 가지 콘솔 게임을 **단계별로** 만들어 본다. 각 게임은 클래스를 하나씩 쌓아가며 완성하는 구조로, 클래스 간 **협력**, **상속**, **오버라이딩**을 체험한다.

- 하나의 게임을 여러 클래스로 나누어 설계할 수 있다.
- 한 클래스가 다른 클래스의 객체를 필드로 가지는 **협력** 구조를 이해한다.
- **상속**으로 기존 클래스를 확장하는 방법을 익힌다.
- **virtual/new**를 이용한 메서드 오버라이딩을 구현할 수 있다.
- GameManager 패턴으로 여러 클래스를 조합하여 게임을 완성할 수 있다.

1.2. 강의 구성

파트	게임	내용
A부	숫자 맞추기	클래스 복습 → 클래스 협력 → 상속/오버라이딩
B부	숫자 야구	배열 필드 → 스트라이크/볼 판정 → 게임 완성
C부	행맨	문자열 검사 → 그림 출력 → 빈칸 채우기 → 게임 완성

클래스 설계의 핵심 원칙

각 게임은 동일한 패턴으로 완성된다:

1. **데이터 클래스** — 게임의 핵심 데이터를 저장하고 검증한다.
2. **Checker 클래스** — 사용자 입력을 받고, 데이터 클래스에 전달하여 결과를 얻는다.
3. **GameManager 클래스** — 위 둘을 조합하여 게임 루프를 진행한다.

main 코드는 항상 GameManager를 생성하고 Play()를 호출하는 2줄로 끝난다!

2. A부: 숫자 맞추기 게임

숫자 맞추기 게임은 컴퓨터가 정한 비밀 숫자를 사용자가 추측하는 게임이다. 세 단계에 걸쳐 클래스를 하나씩 추가하며 게임을 완성한다.

2.1. Step A-1: MyNumber 클래스 만들기

가장 기본적인 클래스를 만든다. 양의 정수 하나를 저장하고, 일치 여부를 검사하는 기능만 가진다.

MyNumber 클래스 설계

구분	이름	설명
필드	private int number	저장된 양의 정수
필드	public int Max	최대값 (읽기 전용)
생성자	MyNumber(int max)	최대값 지정, number는 0으로 초기화
메서드	SetNumber(int i)	1~Max 범위면 저장, 아니면 오류
메서드	CheckNumber(int j)	j와 number가 같으면 true

클래스 정의

```

1 class MyNumber
2 {
3     private int number;           // stored positive integer
4     public int Max { get; }       // max value (read-only)
5
6     public MyNumber(int max)
7     {
8         Max = max;
9         number = 0;               // initial value
10    }
11
12    // Store i if it is in range [1, Max]
13    public void SetNumber(int i)
14    {
15        if (i >= 1 && i <= Max)
16            number = i;
17        else
18            Console.WriteLine(" [Error] " + i
19                               + " is out of range! (1~" + Max + ")");
20    }
21
22    // Return true if j equals number
23    public bool CheckNumber(int j)
24    {
25        return number == j;
26    }
27 }
```

테스트 코드 (main)

```

1 MyNumber myNum = new MyNumber(100);
2
3 myNum.SetNumber(42);
4 Console.WriteLine("CheckNumber(42) -> " + myNum.CheckNumber(42)
5 ); // True
6 Console.WriteLine("CheckNumber(10) -> " + myNum.CheckNumber(10)
7 ); // False
8
9 myNum.SetNumber(0); // Error: out of range
10 myNum.SetNumber(-5); // Error: out of range
11 myNum.SetNumber(200); // Error: out of range

```

private의 역할

number는 private이므로 외부에서 myNum.number = -1; 같은 직접 접근이 불가능하다. 반드시 SetNumber()를 통해서만 값을 설정할 수 있으며, 이때 범위 검증이 자동으로 수행된다. 이것이 **캡슐화(Encapsulation)**의 핵심이다.

2.2. Step A-2: Checker 클래스 — 클래스 간 협력

이제 두 번째 클래스 Checker를 추가한다. Checker는 사용자 입력을 받고, MyNumber에게 전달하여 결과를 얻는 역할을 한다.

클래스 간 협력 구조

```

[사용자] --> Checker.GetInputNumber() --> Checker.Check()
                                     |
                                     MyNumber.CheckNumber()
                                     |
                                     true / false

```

Checker는 생성자에서 MyNumber 객체를 전달받아 필드로 보관한다. 이처럼 한 클래스가 다른 클래스의 객체를 필드로 가지는 것을 **클래스 간 협력**이라 한다.

Checker 클래스 정의

```

1 class Checker
2 {
3     private MyNumber target; // target MyNumber object
4
5     // Constructor: receive MyNumber to check
6     public Checker(MyNumber target)
7     {
8         this.target = target;
9     }
10
11     // Get number input from user (-1 if invalid)

```

```

12     public int GetInputNumber()
13     {
14         Console.WriteLine("Enter number: ");
15         string input = Console.ReadLine();
16
17         int result;
18         if (!int.TryParse(input, out result))
19         {
20             Console.WriteLine(" [Error] Please enter a number!
21             ");
22             return -1;
23         }
24         if (result < 1 || result > target.Max)
25         {
26             Console.WriteLine(" [Error] Enter a number between
27             1~"
28             + target.Max + "!");
29             return -1;
30         }
31         return result;
32     }
33
34     // Pass value to MyNumber and return result
35     public bool Check(int value)
36     {
37         return target.CheckNumber(value);
38     }
39 }

```

테스트 코드 (main)

```

1  MyNumber myNum = new MyNumber(100);
2  myNum.SetNumber(42);
3
4  Checker checker = new Checker(myNum);
5
6  int input = checker.GetInputNumber(); // user input
7  if (input != -1)
8  {
9      bool result = checker.Check(input); // pass to MyNumber
10     Console.WriteLine("Result: " + result);
11 }

```

2.3. Step A-3: 상속과 오버라이딩으로 게임 완성

이제 이번 강의의 핵심 개념인 **상속(Inheritance)**과 **오버라이딩(Overriding)**을 도입한다.

상속과 오버라이딩이란?

- **상속**: 기존 클래스(부모)의 필드와 메서드를 물려받아 새 클래스(자식)를 만드는 것
- **오버라이딩**: 부모가 가진 메서드를 자식이 자기 방식으로 재정의하는 것

MyNumber의 CheckNumber는 bool(true/false)만 반환한다.

GameNumber는 이를 상속받아, "=", "-", "+" 문자열로 더 자세한 힌트를 제공하도록 재정의한다.

부모 클래스: MyNumber에 virtual 추가

```

1 class MyNumber
2 {
3     private int number;
4     public int Max { get; }
5
6     public MyNumber(int max) { Max = max; number = 0; }
7
8     public void SetNumber(int i)
9     {
10         if (i >= 1 && i <= Max) number = i;
11         else Console.WriteLine(" [Error] out of range!");
12     }
13
14     // virtual: child class can override this
15     public virtual bool CheckNumber(int j)
16     {
17         return number == j;
18     }
19
20     public int GetNumber() { return number; }
21 }
```

자식 클래스: GameNumber

```

1 class GameNumber : MyNumber    // MyNumber (inheritance)
2 {
3     // Call parent constructor via base(max)
4     public GameNumber(int max) : base(max) { }
5
6     // Override parent's CheckNumber
7     // "=" match, "-" guess too small, "+" guess too big
8     public new string CheckNumber(int j)
9     {
10         if (j == GetNumber()) return "=";
11         if (j < GetNumber()) return "-";
12         return "+";
13     }
14 }
```

```

13     }
14 }

```

상속 문법 요약

<code>class GameNumber : MyNumber</code>	상속 선언 (GameNumber는 MyNumber의 자식)
<code>: base(max)</code>	생성자에서 부모 생성자 호출
<code>virtual</code>	부모에서 선언: “자식이 재정의해도 좋다”
<code>new</code>	자식에서 선언: “부모 메서드를 새로 정의한다”

GameManager 클래스 — 게임 전체 진행

```

1  class GameManager
2  {
3      private GameNumber secret;
4      private Checker checker;
5
6      public GameManager(int max)
7      {
8          secret = new GameNumber(max);
9          Random rand = new Random();
10         secret.SetNumber(rand.Next(1, max + 1));
11         checker = new Checker(secret);
12     }
13
14     public void Play()
15     {
16         Console.WriteLine("=== Number Guessing Game ===");
17         Console.WriteLine("Guess a number between 1 ~ "
18             + secret.Max + "!");
19         Console.WriteLine("Hint: [+] go lower  [-] go higher\n"
20             );
21
22         int turn = 0;
23         while (true)
24         {
25             int input = checker.GetInputNumber();
26             if (input == -1) continue;
27             turn++;
28
29             string result = checker.Check(input);
30             if (result == "=")
31             {
32                 Console.WriteLine(" * Correct! Congratulations
33                     !");
34                 Console.WriteLine(" * " + turn + " tries!");
35                 break;
36             }
37             else if (result == "-")
38                 Console.WriteLine(" -> Go higher");

```



```

37         else
38             Console.WriteLine("    -> Go lower");
39     }
40 }
41 }

```

main 코드 — 단 2줄!

```

1 GameManager manager = new GameManager(100);
2 manager.Play();

```

상속과 오버라이딩 — 무엇이 달라졌는가?

Step A-3에서 일어난 핵심적인 변화를 정리하면 다음과 같다.

상속 전후 비교: CheckNumber 메서드

A-1 (부모: MyNumber) — 일치 여부만 반환

```

CheckNumber(42) ⇒ true
CheckNumber(10) ⇒ false

```

문제: false만으로는 “더 큰 수인지, 더 작은 수인지” 알 수 없다.

A-3 (자식: GameNumber) — 힌트까지 포함하여 반환

```

CheckNumber(42) ⇒ "=" (일치)
CheckNumber(10) ⇒ "-" (입력이 더 작음 → 더 큰 수를 시도)
CheckNumber(99) ⇒ "+" (입력이 더 큼 → 더 작은 수를 시도)

```

부모의 코드를 한 줄도 수정하지 않고, 자식 클래스에서 메서드를 재정의하여 기능을 확장했다!

왜 상속을 사용하는가?

1. **코드 재사용**: GameNumber는 SetNumber, GetNumber, Max 등 부모의 모든 기능을 그대로 물려받는다. 다시 작성할 필요가 없다.
2. **기존 코드 보호**: 부모 MyNumber는 변경되지 않으므로, A-1, A-2에서 이미 테스트된 코드가 깨지지 않는다.
3. **기능 확장**: 자식은 CheckNumber만 재정의하여 “=/-/+” 힌트 기능을 추가했다. 부모에는 없던 새 기능이다.

이것이 객체지향의 **개방-폐쇄 원칙**이다: “확장에는 열려 있고(자식 추가 가능), 수정에는 닫혀 있다(부모 변경 불필요).”

virtual과 new 키워드 상세 설명

- **virtual** (부모에 선언): “이 메서드는 자식 클래스가 재정의할 수 있다”는 **허가**를 의미한다. **virtual**이 없으면 자식이 재정의할 수 없다.
- **new** (자식에 선언): “부모의 메서드를 숨기고 새로운 버전을 정의한다”는 의미이다. 이번 예제에서는 반환 타입이 **bool** → **string**으로 달라졌기 때문에 **new**를 사용했다.
- **override** (참고): 반환 타입이 같을 때는 **new** 대신 **override**를 사용한다. **override**는 부모 타입 변수로 호출해도 자식 메서드가 실행되는 **다형성(Polymorphism)**을 지원한다.

실습 A: 숫자 맞추기 게임

1. StepA_1, StepA_2, StepA_3을 순서대로 실행해 보자.
2. A-3에서 범위를 1000으로 바꿔보고, 몇 번 만에 맞출 수 있는지 확인하자.
3. GameNumber의 CheckNumber에서 “차이가 10 이하”일 때 “거의 다 왔습니다!” 힌트를 추가해 보자.

3. B부: 숫자 야구 게임

숫자 야구는 비밀 세 자리 숫자(1~9, 중복 없음)를 맞추는 게임이다. 같은 자리 · 같은 숫자면 **스트라이크(S)**, 다른 자리 · 같은 숫자면 **볼(B)**이다.

3.1. Step B-1: SecretNum 클래스

세 자리 숫자를 `int[]` 배열로 저장하며, `SetNumber`에서 1~9 범위와 중복 여부를 검증한다.

```
1 class SecretNum
2 {
3     private int[] digits;
4
5     public SecretNum()
6     {
7         digits = new int[3];
8     }
9
10    // Range 1~9 + no duplicates -> store and return true
11    public bool SetNumber(int a, int b, int c)
12    {
13        if (a < 1 || a > 9 || b < 1 || b > 9 || c < 1 || c > 9)
14        {
15            Console.WriteLine(" [Error] Only 1~9 allowed!");
16            return false;
17        }
18        if (a == b || b == c || a == c)
19        {
20            Console.WriteLine(" [Error] Duplicate number!");
21            return false;
22        }
23        digits[0] = a; digits[1] = b; digits[2] = c;
24        return true;
25    }
26
27    public int GetDigit(int index) { return digits[index]; }
28
29    public string GetNumber()
30    {
31        return digits[0] + " " + digits[1] + " " + digits[2];
32    }
33 }
```

검증의 중요성

`SetNumber`가 실패(false)하면 기존 값이 유지된다. 잘못된 데이터가 저장되는 것을 클래스 내부에서 차단하는 것이 핵심이다. 외부에서는 반환된 `bool` 값만 확인하면 된다.

3.2. Step B-2: Checker 클래스 — 스트라이크/볼 판정

Checker는 사용자에게 숫자 3개를 입력받고, SecretNum.GetDigit()을 이용하여 스트라이크/볼을 판정한다.

판정 로직

```

1 // Check method: Strike/Ball judgment
2 public int[] Check(int a, int b, int c)
3 {
4     int[] guess = { a, b, c };
5     int strike = 0;
6     int ball = 0;
7
8     for (int i = 0; i < 3; i++)
9     {
10        for (int j = 0; j < 3; j++)
11        {
12            if (guess[i] == target.GetDigit(j))
13            {
14                if (i == j) strike++; // same position -> S
15                else ball++; // different position ->
16                               B
17            }
18        }
19        return new int[] { strike, ball };
20    }

```

판정 예시

비밀 숫자: 3 8 5

입력	비교	Strike	Ball	결과
3 8 5	전부 일치	3	0	3S 0B (정답!)
3 5 1	3은 제자리, 5는 다른 자리	1	1	1S 1B
5 3 8	모두 있지만 자리 다름	0	3	0S 3B
1 2 4	아무것도 없음	0	0	Out!

3.3. Step B-3: GameManager로 게임 완성

GameManager가 SecretNum(랜덤 생성)과 Checker를 조합하여 3S가 나올 때까지 반복하는 게임을 완성한다.

```

1 class GameManager
2 {
3     private SecretNum secret;
4     private Checker checker;
5
6     public GameManager()
7     {

```

```

8      secret = new SecretNum();
9      // Pick 3 unique numbers from 1~9
10     Random rand = new Random();
11     List<int> pool = new List<int>
12         { 1, 2, 3, 4, 5, 6, 7, 8, 9 };
13     int a = pool[rand.Next(pool.Count)]; pool.Remove(a);
14     int b = pool[rand.Next(pool.Count)]; pool.Remove(b);
15     int c = pool[rand.Next(pool.Count)];
16     secret.SetNumber(a, b, c);
17     checker = new Checker(secret);
18 }
19
20 public void Play()
21 {
22     Console.WriteLine("=== Number Baseball ===");
23     Console.WriteLine("Guess 3 unique digits from 1~9!\n");
24     int turn = 0;
25     while (true)
26     {
27         int[] input = checker.GetInputNumbers();
28         if (input == null) continue;
29         turn++;
30
31         int[] result = checker.Check(
32             input[0], input[1], input[2]);
33         int strike = result[0];
34         int ball = result[1];
35
36         if (strike == 3)
37         {
38             Console.WriteLine(" * 3S! Correct!");
39             Console.WriteLine(" * " + turn
40                 + " tries!");
41             break;
42         }
43         else if (strike == 0 && ball == 0)
44             Console.WriteLine(" -> Out!");
45         else
46             Console.WriteLine(" -> " + strike
47                 + "S " + ball + "B");
48     }
49 }
50 }

```

main 코드 — 역시 2줄!

```

1  GameManager manager = new GameManager();
2  manager.Play();

```

실습 B: 숫자 야구 게임

1. StepB_1, StepB_2, StepB_3을 순서대로 실행해 보자.
2. B-1에서 `SetNumber(1,0,3)`, `SetNumber(3,3,5)` 등 실패 케이스를 테스트하자.
3. 4자리 숫자 야구로 확장하려면 어떤 클래스를 수정해야 할지 생각해 보자.

4. C부: 행맨 (Hangman) 게임

행맨은 비밀 단어의 알파벳을 한 글자씩 맞추는 게임이다. 틀릴 때마다 목숨이 줄어들고, 목숨이 0이 되면 게임 오버다. 네 단계에 걸쳐 각각 다른 역할의 클래스를 만들고, 마지막에 조합한다.

4.1. Step C-1: WordClass — 단어 저장과 알파벳 검사

WordClass는 단어를 저장하고, 특정 알파벳이 몇 개 있으며, 어디에 있는지를 배열로 반환한다.

```
1 class WordClass
2 {
3     private string word;
4
5     public WordClass() { word = ""; }
6
7     public void SetString(string s) { word = s.ToUpper(); }
8     public string GetWord() { return word; }
9     public int GetLength() { return word.Length; }
10
11     // Return positions of char c as int array
12     public int[] CheckString(char c)
13     {
14         c = char.ToUpper(c);
15
16         // Step 1: count occurrences
17         int count = 0;
18         for (int i = 0; i < word.Length; i++)
19             if (word[i] == c) count++;
20
21         // Step 2: build position array
22         int[] positions = new int[count];
23         int idx = 0;
24         for (int i = 0; i < word.Length; i++)
25         {
26             if (word[i] == c)
27             {
28                 positions[idx] = i;
29                 idx++;
30             }
31         }
32         return positions;
33     }
34 }
```

CheckString 동작 예시

단어: BANANA

호출	반환	의미
CheckString('A')	int[] {1, 3, 5}	3개, 인덱스 1·3·5
CheckString('N')	int[] {2, 4}	2개, 인덱스 2·4
CheckString('B')	int[] {0}	1개, 인덱스 0
CheckString('Z')	int[] {}	0개, 빈 배열

반환된 배열의 .Length가 개수이고, 배열 내용이 위치이다.

4.2. Step C-2: HangmanDrawing — 그림 그리기

HangmanDrawing은 남은 목숨(life)에 따라 행맨 그림을 출력한다. 목숨은 8에서 시작하며, 0이 되면 모든 신체 부위가 완성(사망)된다.

```

1 class HangmanDrawing
2 {
3     // stages[0]=life 0(dead) ~ stages[8]=life 8(start)
4     private string[][] stages = new string[][] { ... };
5
6     public void Draw(int life)
7     {
8         if (life < 0) life = 0;
9         if (life > 8) life = 8;
10        foreach (string line in stages[life])
11            Console.WriteLine(line);
12    }
13 }
```

목숨에 따른 그림 변화

life 8 (시작)	life 5	life 0 (사망)
+---+	+---+	+---+
	0	0
		* * Failed!
		/ \
+-----	+-----	+-----

틀릴 때마다 life가 1 감소하며 신체 부위가 추가된다:

머리(O) → 몸통(l) → 오른팔(\) → 왼손(*) → 오른손(*) → 왼다리(/) → 오른다리(\)

4.3. Step C-3: CheckWord — 빈칸 관리와 글자 채우기

CheckWord는 WordClass를 이용하여 빈칸 배열(.)을 관리하고, 사용자의 입력에 따라 글자를 채운다.

```

1 class CheckWord
2 {
3     private WordClass target;
4     private char[] display;    // '_' or letter
5 }
```



```

6     public CheckWord(WordClass target)
7     {
8         this.target = target;
9         display = new char[target.GetLength()];
10        for (int i = 0; i < display.Length; i++)
11            display[i] = '_';
12    }
13
14    // Get one letter from user
15    public char GetInputChar() { ... }
16
17    // Fill matched positions with the letter
18    public bool Check(char c)
19    {
20        int[] positions = target.CheckString(c);
21        if (positions.Length == 0) return false;
22
23        for (int i = 0; i < positions.Length; i++)
24            display[positions[i]] = c;
25        return true;
26    }
27
28    // Return current state like "B _ N _ N _"
29    public string GetDisplay() { ... }
30
31    // Check if all blanks are filled
32    public bool IsComplete() { ... }
33 }

```

CheckWord 동작 흐름

단어: BANANA

시점 입력 display 상태

처음	—	_ _ _ _ _
1회	B	B _ _ _ _
2회	A	B A _ A _ A
3회	X	B A _ A _ A (없는 글자 → 목숨 감소)
4회	N	B A N A N A → 완성!

4.4. Step C-4: GameManager로 행맨 완성

GameManager가 C-1, C-2, C-3의 세 클래스를 모두 조합하여 게임을 완성한다.

```

1     class GameManager
2     {
3         private WordClass word;
4         private HangmanDrawing drawing;
5         private CheckWord checker;
6         private int life;
7     }

```

```
8     private static string[] wordList =
9         { "APPLE", "BANANA", "CHERRY", "DRAGON",
10           "FOREST", "GUITAR", "HAMMER", "ISLAND",
11           "JUNGLE", "KNIGHT" };
12
13     public GameManager()
14     {
15         word = new WordClass();
16         string picked =
17             wordList[new Random().Next(wordList.Length)];
18         word.SetString(picked);
19
20         drawing = new HangmanDrawing();
21         checker = new CheckWord(word);
22         life = 8;
23     }
24
25     public void Play()
26     {
27         Console.WriteLine("=== Hangman ===");
28         Console.WriteLine("Guess the word letter by letter!");
29         Console.WriteLine("Lives: " + life + "\n");
30
31         int turn = 0;
32         while (life > 0 && !checker.IsComplete())
33         {
34             drawing.Draw(life);           // Draw hangman
35             Console.WriteLine();
36             Console.WriteLine("  Word: "
37                 + checker.GetDisplay()); // Show blanks
38             Console.WriteLine("  Lives: " + life);
39             Console.WriteLine();
40
41             char input = checker.GetInputChar();
42             if (input == ' ') continue;
43             turn++;
44
45             bool hit = checker.Check(input);
46             if (hit)
47                 Console.WriteLine("    -> Correct!\n");
48             else
49             {
50                 life--;
51                 Console.WriteLine("    -> Wrong!\n");
52             }
53         }
54
55         // Final result
56         drawing.Draw(life);
57         Console.WriteLine("\n  Word: "
58             + checker.GetDisplay() + "\n");
```

```
59
60     if (checker.IsComplete())
61     {
62         Console.WriteLine(" * Congratulations! Answer: "
63             + word.GetWord());
64         Console.WriteLine(" * " + turn
65             + " tries!");
66     }
67     else
68         Console.WriteLine(" Game Over! Answer: "
69             + word.GetWord());
70 }
71 }
```

main 코드 — 동일한 패턴 2줄

```
1 GameManager manager = new GameManager();
2 manager.Play();
```

실습 C: 행맨 게임

1. StepC_1~StepC_4를 순서대로 실행해 보자.
2. C-1에서 다른 단어(예: PROGRAMMING)로 CheckString을 테스트하자.
3. C-4에서 단어 목록에 5개의 단어를 추가해 보자.
4. 이미 시도한 글자를 다시 입력하면 “이미 시도한 글자입니다”라고 표시하도록 수정하자.

5. 상속과 다형성 — 세 게임을 관통하는 핵심 원리

이번 강의의 세 게임에서 상속과 다형성이 어떻게 활용되었는지 종합적으로 살펴본다.

5.1. 상속이 사용된 곳: Step A-3의 GameNumber

A부에서 유일하게 상속이 등장한다. GameNumber가 MyNumber를 상속받아 CheckNumber를 재정의한 것이 핵심이다.

	부모: MyNumber	자식: GameNumber
필드	private int number public int Max	(상속받아 그대로 사용) (상속받아 그대로 사용)
메서드	SetNumber(int i) GetNumber() virtual CheckNumber(int j) → bool	(상속받아 그대로 사용) (상속받아 그대로 사용) new CheckNumber(int j) → string
새로 작성한 코드	—	CheckNumber 1개만 재정의

자식 클래스 GameNumber는 **메서드 1개만 새로 작성**하고, 나머지 4개(number, Max, SetNumber, GetNumber)는 부모에게서 그대로 물려받았다. 이것이 상속의 **코드 재사용** 효과이다.

5.2. 클래스 간 협력이 사용된 곳: 모든 게임

세 게임 모두 “한 클래스가 다른 클래스의 객체를 필드로 보관”하는 협력 구조를 사용한다. 이것은 상속과는 다른 형태의 클래스 관계이다.

상속 vs 협력(컴포지션)

- **상속** (class B : A): B 는 A이다 (is-a 관계)
 - GameNumber는 MyNumber**이다** → MyNumber의 모든 기능을 가진다
- **협력/컴포지션** (필드로 보관): B 가 A를 가진다 (has-a 관계)
 - Checker가 MyNumber를 가진다 → MyNumber에게 일을 시킨다
 - GameManager가 Checker와 데이터 클래스를 가진다

각 게임의 협력 구조를 그림으로 나타내면:

세 게임의 클래스 관계도

[A부: 숫자 맞추기]

MyNumber <--- GameNumber (상속)

Checker (협력: GameNumber를 필드로 보관)

GameManager (협력: GameNumber + Checker 보관)

```

[B부: 숫자 야구]
SecretNum
|
Checker (협력: SecretNum을 필드로 보관)
|
GameManager (협력: SecretNum + Checker 보관)

[C부: 행맨]
WordClass
|
CheckWord (협력: WordClass를 필드로 보관)
|
GameManager (협력: WordClass + CheckWord + HangmanDrawing 보관)

```

5.3. 다형성의 기초: “같은 이름, 다른 동작”

다형성(Polymorphism)이란 “같은 메서드 호출이 객체에 따라 다르게 동작”하는 것이다. 이번 강의에서는 다음 두 가지 형태로 다형성이 나타난다.

1. 메서드 재정의에 의한 다형성 (A-3)

MyNumber와 GameNumber는 모두 CheckNumber라는 같은 이름의 메서드를 가지지만, 동작이 다르다:

객체 타입	CheckNumber(42) 호출	반환값
MyNumber 객체	일치 여부만 판단	true (bool)
GameNumber 객체	힌트까지 포함	"=" (string)

같은 이름 CheckNumber를 호출하지만, 객체의 타입에 따라 전혀 다른 결과가 나온다. 이것이 다형성의 핵심이다.

2. 같은 역할, 다른 구현 — GameManager 패턴 (전체)

세 게임의 GameManager는 모두 Play() 메서드를 가지며, 모두 “입력 → 판정 → 결과 출력 → 반복”이라는 동일한 구조를 따른다. 하지만 내부 구현은 게임마다 완전히 다르다:

	숫자 맞추기	숫자 야구	행맨
입력	숫자 1개	숫자 3개	알파벳 1개
판정	“=”/“−”/“+”	Strike/Ball	위치 배열
종료 조건	“=” 반환 시	3S 시	빈칸 완성 또는 목숨 0

만약 세 GameManager를 하나의 부모 클래스 Game에서 상속받도록 만든다면, Game 타입의 변수 하나로 어떤 게임이든 실행할 수 있게 된다. 이것이 다음 강의에서 다룰 **추상 클래스**와 **인터페이스**의 출발점이다.

생각해 보기: 다형성 확장

현재 세 게임의 GameManager는 각각 독립적인 클래스이다. 만약 다음과 같은 코드가 가능하려면 어떻게 설계해야 할까?

```
1 Game game; // parent type
2 game = new NumberGuessGame(100); // child 1
3 game.Play();
4
5 game = new BaseballGame(); // child 2
6 game.Play();
7
8 game = new HangmanGame(); // child 3
9 game.Play();
```

힌트: Game 부모 클래스에 virtual void Play()를 선언하고, 각 게임이 이를 override하면 된다!

6. 핵심 개념 정리

6.1. 이번 강의에서 다룬 OOP 개념

개념	설명
캡슐화	private 필드를 메서드로만 접근하여 잘못된 값 차단
클래스 간 협력	한 클래스가 다른 클래스의 객체를 필드로 보관하여 역할 분담
상속	class 자식 : 부모로 기존 클래스의 기능을 물려받음
오버라이딩	virtual/new로 부모 메서드를 자식이 재정의
GameManager 패턴	여러 클래스를 조합하여 게임 루프를 관리하는 클래스

6.2. 세 게임의 공통 구조

역할	숫자 맞추기	숫자 야구	행맨
데이터	MyNumber	SecretNum	WordClass
판정/입력	Checker	Checker	CheckWord
출력	—	—	HangmanDrawing
게임 관리	GameManager	GameManager	GameManager
main 코드	2줄	2줄	2줄

게임은 달라도 구조는 같다. 이것이 클래스 기반 설계의 가장 큰 장점이다.

도전 과제

- 숫자 야구 확장:** 4자리 숫자 야구로 수정하라. SecretNum, Checker, GameManager 중 어떤 클래스를 수정해야 하는가?
- 행맨 난이도:** “쉬움(life 10)”, “보통(life 8)”, “어려움(life 5)” 중 선택하는 기능을 추가하라.
- 새 게임:** 가위바위보 게임을 같은 구조(데이터 클래스 + Checker + GameManager)로 만들어 보라.